AI Workforce Advisor: A Machine Learning Approach to Predicting AI-Driven Productivity Gains

Ritik Mehra
Roll Number: 23035010235, Term Project Report — Trimester IX (DA377)
B.Sc. (Honours) Data Science and Artificial Intelligence
Indian Institute of Technology Guwahati, India
ritik.mehra@op.iitg.ac.in

Abstract

A lot of people use AI tools like ChatGPT, Claude, or GitHub Copilot every day at work, but very few of them actually know how much these tools are helping them. This project tries to answer that question by building a machine learning model that predicts a person's expected productivity gain from using AI, based on just six simple things a person can tell about themselves: their industry, job role, years of experience, the AI tool they mainly use, how often they use AI in a day, and how many tasks they automate every week. I used a dataset with 15,000 user records to train and compare two models, Linear Regression and Random Forest. Random Forest gave much better results, with an R^2 of 0.874 and an average error of only 2.84 percentage points, compared to Linear Regression's R^2 of 0.80. Looking at feature importance, I found something interesting: how often someone uses AI and automates tasks matters far more than their job, industry, or which AI tool they pick. I also ran a small experiment to check if a user's location was actually needed, and found that removing it barely changed the accuracy, so I dropped it from the final model. In the end, the model was turned into a Streamlit web app that shows the predicted productivity gain, an AI adoption score, time saved, and some simple tips, and I deployed it online so anyone can try it.

1 Introduction

AI tools are now part of daily work for a huge number of people, whether it is a software developer using a code assistant or a marketer writing content with AI help. But even though people use these tools all the time, most of them cannot really say how much AI is actually improving their work. Companies face a similar issue too, they spend money on AI tools without a clear, data-backed way to see if it is paying off.

This project tries to fix that in a small way. I built a tool called the AI Workforce Advisor, which asks a person a few easy questions about their job and AI habits, and in return gives them a predicted productivity gain, an AI adoption score, an estimate of time saved, and a few personalised suggestions.

The rest of the report is organised like this. Section II explains the problem and what I was trying to achieve. Section III walks through everything I did, from exploring the data to deploying the app. Section IV shows the results. Section V talks about what the results actually mean and where the model falls short. Section VI wraps up with a conclusion and ideas for future work, and Section VII has the public links to the project.

2 Problem Statement and Objectives

2.1 Problem Statement

This is a regression problem. Given some basic information about a person's job and how they use AI, the goal is to predict their `Productivity_Gain_Percent`, which is just a number showing how much their output improved because of AI.

The dataset I used is a user-level AI adoption dataset [1] with 15,000 rows and 10 original columns. It has a user ID, some work-related columns, some AI usage columns, a date, and the target column I am trying to predict.

2.2 Objectives

Here is what I set out to do in this project:

1. Explore and clean the data so it is ready to use.
2. Only keep features that a real person could actually answer honestly.
3. Train and compare two different regression models.
4. Check whether removing the location column actually hurts accuracy before deciding to drop it.
5. Find out which features actually matter the most for the prediction.
6. Turn the final model into a working web app that anyone can access online.

3 Methodology

3.1 Workflow Overview

I followed a fairly simple, step-by-step process for this project: load the data, explore it, clean it, pick and engineer the right features, encode everything, split it into train and test sets, train both models, check the results, and finally deploy it. Each step depended on the one before it.

3.2 Data Loading and Exploration

I loaded the CSV file using Pandas [4] and used basic commands like `.shape`, `.info()`, `.isnull().sum()`, and `.duplicated().sum()` to get a first look at the data. There were no missing values and no duplicate rows across all 15,000 records, which was a relief since it meant I did not have to spend extra time fixing broken data. Fig. 1 shows how the target column, `Productivity_Gain_Percent`, is spread out before I did any modelling. Most people show a low to medium gain, and only a small number of users show a really high gain, which actually makes sense, since usually only a few "power users" get the most out of any tool.

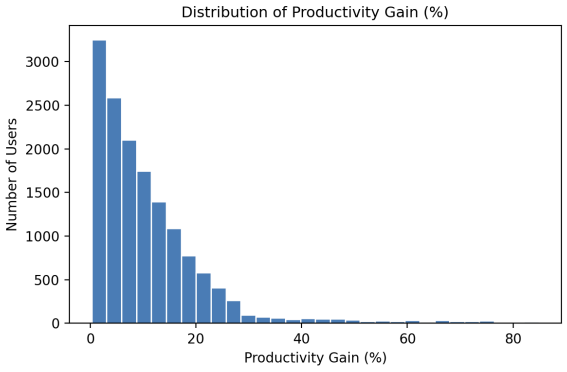

Figure 1: Distribution of Productivity Gain across all users.

3.3 Data Cleaning

Since the previous step already showed no missing values and no duplicates, this step did not involve fixing anything. I still ran the cleaning checks properly instead of just assuming the data was fine, because in a real project you should always confirm this rather than guess.

3.4 Feature Engineering: Adoption Date

I pulled out the `Year` and `Month` from the `Adoption_Date` column and checked if they had any real relationship with productivity gain. The average gain barely moved across different years (around 11.03% to 11.36%) or months (around 10.48% to 11.54%), so there was basically no pattern there. Because of this, I did not use `Year` or `Month` as features, and I also removed `User_ID` since it is just an identifier and has nothing to do with the prediction.

3.5 Feature Selection Experiment: Is Location Necessary?

The dataset also had a `Location` column, showing which region the user was from. Asking a person their location felt like an unnecessary and slightly awkward question for this kind of app, so I decided to actually test whether it was needed instead of just guessing. I trained the same Random Forest model once with `Location` included, and once without it, keeping everything else exactly the same. Table I shows what happened.

Table 1: Effect of Removing the Location Feature

Feature Set	MAE	RMSE	R^2
All 7 features (incl. Location)	2.82	4.16	0.877
Final 6 features (no Location)	2.84	4.20	0.874

The difference was only 0.003 in R^2, which is basically nothing. Since location barely affected the prediction and removing it made the app simpler and less invasive to use, I decided to leave it out of the final model for good. So the six features I ended up using were: Industry, Job Role, Experience (Years), Primary AI Tool, Daily AI Usage, and Tasks Automated Per Week. These are all things a person can just answer directly, without needing to look anything up.

3.6 Encoding and Train-Test Split

I converted the categorical columns (Industry, Job Role, Primary AI Tool) into numbers using One-Hot Encoding instead of Label Encoding. I chose One-Hot Encoding on purpose, because Label Encoding would have accidentally made it look like one category is "bigger" than another, which does not make sense for something like industry names. After that, I split the data into 80% for training and 20% for testing, using a fixed random seed so the same results can be reproduced later.

3.7 Model Training

I trained two models on the same training data, and kept both of them simple so I could actually explain every decision if asked.

The first one, Linear Regression, is the simplest option. It basically tries to draw a straight-line relationship between the inputs and the output:

$$\hat{y} = w_0 + \sum_{i=1}^{n} w_i x_i \tag{1}$$

Here $\hat{y}$ is the predicted productivity gain, x_i are the encoded input values, and w_i are the weights the model learns while training.

The second model, Random Forest Regression [3], is a bit more powerful. I used the version from Scikit-learn [2]. Instead of one straight line, it builds a lot of small decision trees on random parts of the data, and then averages what they all

say. This lets it pick up on patterns that are not just a straight line, for example, someone who uses AI heavily and also automates a lot of tasks might get a much bigger boost than either factor alone would suggest. I did not use any automated tuning tool like GridSearchCV here. I just went with a simple setup of 100 trees, mainly so every choice in the model stays easy to explain during the viva.

3.8 Model Deployment

Once the model was trained, I saved both the model and the fitted encoder using joblib. Then I built a Streamlit web app that loads these files and shows a sidebar with the six inputs. One thing I changed here: instead of asking the user for their exact daily token usage (which basically nobody actually knows), I replaced it with a simple question, "How much do you use AI tools daily?", with five options ranging from Rarely to Extremely heavily. Each option is quietly mapped to a realistic number behind the scenes before being sent to the model, so the model still gets the same kind of numeric input it was trained on, but the user only has to pick something they can honestly answer. When the Predict button is clicked, the app puts all the inputs together, runs it through the model, and shows the predicted productivity gain along with an AI Adoption Score, an estimate of time saved, and a few personalised recommendations.

4 Experiments and Results

4.1 Experimental Setup

- Dataset: user-level AI adoption dataset, 15,000 records, nothing needed to be removed during cleaning.
- Split: 80% train / 20% test, fixed random seed.
- Environment: Python 3, Pandas, Scikit-learn, Streamlit.
- Models: Linear Regression and Random Forest Regression.
- Metrics: R^2, MAE, RMSE.

4.2 Model Performance

Table II shows how both models did on the test set. MAE and RMSE tell us, roughly, how many percentage points off a prediction usually is, so lower is better here. R^2 tells us how much of the variation in productivity gain the model is able to explain, and closer to 1.0 is better.

Table 2: Test-Set Performance of Both Models

Model	MAE	RMSE	R^2
Linear Regression	3.79	5.30	0.80
Random Forest	2.84	4.20	0.87

Random Forest wins on every single metric here. This makes sense to me, since productivity gain probably depends on how different features combine together, for example experience and tool choice working together, rather than each feature acting completely on its own. That is exactly the kind of pattern a straight-line model like Linear Regression struggles to catch.

4.3 Feature Importance

Random Forest also tells us how much each feature actually mattered for its predictions. Fig. 2 shows this. `Tasks_Automated_Per_Week` came out as the most important feature by a large margin, with an importance of 0.62, and `Daily_Token_Usage` came second at 0.32. Together, these two usage-related features make up about 93% of everything the model is basing its prediction on, while Experience, Job Role, Industry, and AI Tool combined add up to less than 7%.

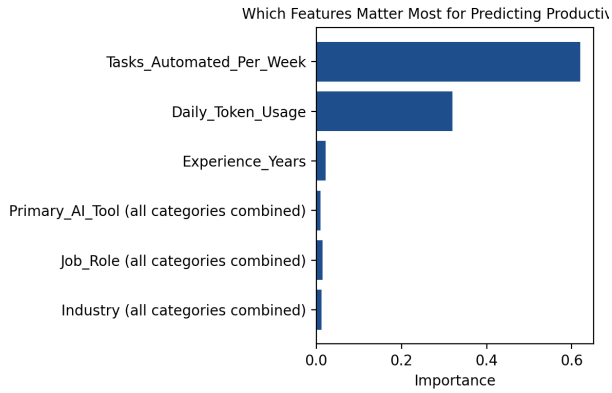

Figure 2: Feature importance scores from the final Random Forest model.

4.4 Cross-Validation

To make sure my R^2 score was not just a lucky result from one particular split of the data, I also ran 5-fold cross-validation on the whole dataset. The average R^2 across the 5 folds came out to 0.882, with a very small standard deviation of 0.004. This tells me the model's performance is pretty stable and not just something that happened to work well once.

5 Discussion

An R^2 of 0.874 basically means that around seven out of every eight percentage points of difference in productivity gain between users can be explained using just six simple inputs. The gap between Linear Regression (0.80) and Random Forest (0.874) tells me that productivity gain is not a simple straight-line relationship, it depends on how different factors combine, and that is exactly what Random Forest is good at capturing.

I think the feature importance result in Section IV-C is probably the most useful thing I found in this whole project. It shows that *how much* someone uses AI and automates their

work matters way more than *which* tool they use, what industry they are in, or how experienced they are. This is honestly a bit surprising, because a lot of people assume that picking the "right" AI tool is what matters most, but the data here says the opposite, it is really about consistent usage.

The location experiment in Section III-E is also worth mentioning. It shows that a decision I made to keep the app simpler and more privacy-friendly did not end up hurting accuracy, and importantly, I actually tested this instead of just assuming it would be fine.

Limitations:

- **Self-reported usage:** The "Daily AI Usage" question in the app is a simple label chosen by the user, mapped to an estimated number, not an exact measured value, so individual predictions can be slightly off because of this.
- **Snapshot data:** This dataset only shows a single point in time. Since AI tools and how people use them change fast, this model would probably need to be retrained after a year or so to stay accurate.
- **Rule-based scoring on top:** The AI Adoption Score and the recommendations shown in the app are based on simple rules I wrote myself, not something the model learned from data. I kept this transparent in the code on purpose, but it is worth being clear that this part is not a machine-learned result.
- **Basic tuning only:** I did not run any automated hyperparameter search, just a standard 100-tree Random Forest, mainly to keep everything easy to explain in the viva.

6 Conclusion and Future Work

In this project, I built a machine learning model that predicts AI-driven productivity gain using 15,000 real user records. I made sure to keep the input features down to just six simple, honest questions, and after comparing two models, Random Forest came out on top with an R^2 of 0.874, clearly better than Linear Regression's 0.80. I also ran a feature-selection experiment which showed that a user's location could be safely dropped without really hurting accuracy, which made the final app simpler and more respectful of user privacy. On top of that, the feature importance results showed that how someone actually uses AI matters a lot more than their job context or which tool they pick. Finally, I deployed the model as a public web app that shows the predicted productivity gain, an AI adoption score, time saved, and some personalised tips.

For future work, it would be good to collect a bigger and more recent dataset, since AI usage patterns keep changing quickly. It would also help to replace the self-reported usage question with real, automatically tracked usage data if that ever becomes available. Another idea would be to let users see how their predicted gain compares to others in the same job role or industry.

7 Artifacts and Demonstrations

All the project files and links are available here:

- **Code Repository:** https://github.com/mehraritik2004-a11y/AI-Workforce-Advisor
- **Live Web Application:** https://ai-workforce-advisor-dvhmfrniwyxb7hhedleizs.streamlit.app/
- **Video Walkthrough:** https://youtu.be/BS3t-olRT5k

References

[1] Shree, "AI Tools Usage and Workplace Productivity Dataset," Kaggle, [Online]. Available: https://www.kaggle.com/datasets/shree0910/ai-tools-usage-and-workplace-productivity-dataset.

[2] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[4] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.

[5] Streamlit Inc., "Streamlit — The fastest way to build and share data apps," [Online]. Available: https://streamlit.io.

[6] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2009.